

Experiment Log

Chronological record of every experiment, decision, and failure mode encountered while building the paragraph paraphrase system. Companion document to [REPORT.md](#) (which presents the final findings); this file shows the path that got us there.

Table of contents

1. [Environment setup & infrastructure issues](#)
2. [Dataset preparation](#)
3. [T5-small fine-tuning](#)
4. [Pegasus fine-tuning attempts \(failed\)](#)
5. [Inference pipeline development](#)
6. [Quantization investigation](#)
7. [CPU latency benchmark](#)
8. [Paragraph-level metrics](#)
9. [LLM API comparison \(Gemini 2.5 Flash\)](#)
10. [API deployment](#)
11. [Cumulative experiment table](#)
12. [Lessons learned](#)

1. Environment setup & infrastructure issues

Experiment 1.1: Native arm64 Python

Goal: Get a working PyTorch + transformers stack on Apple M5.

Setup: System python3 (3.12.2) was Intel x86_64 running under Rosetta. First `pip install torch` only saw versions $\leq 2.2.x$ because PyTorch ≥ 2.3 dropped x86_64 macOS wheels.

Symptom: transformers required `torch $\geq$ 2.4`; installed `torch==2.2.2` silently $\rightarrow$ transformers disabled PyTorch and raised `ImportError: AutoModelForSeq2SeqLM requires the PyTorch library`.

Diagnosis: `arch` $\rightarrow$ arm64, but `python3 -c "import platform; print(platform.machine())"` $\rightarrow$ x86_64. The shell was arm64, the Python was not.

Fix: Recreated the venv with `/opt/homebrew/bin/python3.12` (native arm64). Suddenly all modern torch wheels were available.

Takeaway: On Apple Silicon, **always verify `platform.machine()` inside the Python you're about to use**. Don't trust `arch` from the shell.

Experiment 1.2: iCloud Drive corrupting the venv

Goal: Install full training stack into a stable venv.

Symptom: After multiple `pip install --force-reinstall` cycles, packages started failing with `ImportError: cannot import name 'set_module' from 'numpy._utils' (unknown location)`. Investigation showed `numpy/_utils/` only contained `__pycache__/`, with no `.py` source files. Same pattern across `torch`, `transformers`, `filelock`, `absl-py`.

Diagnosis: Project lived under `~/Documents/telus_assessment`. macOS iCloud Drive syncs `~/Documents` by default. When `pip` extracted wheels, iCloud sometimes intercepted, leaving conflict-renamed shadows (`file 2.pyc`, `file 3.pyc`) and silently skipping the canonical files. Confirmed via `find venv -name "[0-9].pyc"` returning hundreds of ghosts.

Tried first (failed): Delete ghost files in place, force-reinstall `pip` and `torch`. iCloud re-corrupted the *new* installs as they happened.

Final fix: Created the venv outside iCloud:

```
/opt/homebrew/bin/python3.12 -m venv ~/.venvs/telus_assessment
ln -s ~/.venvs/telus_assessment venv
```

Project code stays in `~/Documents/`; venv is symlinked out. Zero recurrences after this.

Takeaway: **Never put a venv under iCloud-synced directories.** This cost ~90 minutes of intermittent debugging; the symptom pattern (random package files missing after install) is easy to misattribute to `pip` bugs.

2. Dataset preparation

Experiment 2.1: `humarin/chatgpt-paraphrases` flatten

Goal: Convert raw dataset into (source, target) sentence pairs for `seq2seq`.

Raw shape: 419,197 rows × 5 paraphrases each, where paraphrases is a *stringified Python list* in the column.

Implementation (`src/prepare_data.py`):

1. `ast.literal_eval()` the paraphrases column to get a real list.
2. Emit one (source=text, target=paraphrase) row per item — 5× expansion.
3. 2% deterministic train/val split (seed 42).

Result: 2,095,985 sentence pairs → 2,053,065 train + 42,920 val.

Smoke validation: Ran with `--max-rows 100` first → 500 pairs, output paths written successfully, example row inspected manually.

Takeaway: Dataset preparation is the right place to do explicit smoke validation. A 30-second smoke test caught the "stringified list" issue before we spent compute on garbage data.

3. T5-small fine-tuning

Experiment 3.1: Hyperparameter selection

Constraints:

- 24 GB M5 unified memory
- MPS only (CUDA absent on Mac)
- fp16 known to produce silent NaN losses on MPS for seq2seq

Choices:

Knob	Value	Why
Base model	T5-small (60M)	Trains in ≤ 3 h on Mac; fits CPU latency budget
Train samples	100,000	Smaller than full 2M to keep epoch < 1 h
Epochs	3	Common heuristic for seq2seq fine-tuning
Batch	$8 \times$ grad-accum 4	Effective batch 32 within MPS memory
LR	$3e-4$, linear schedule	T5 paper default
Precision	fp32	fp16 unstable on MPS
Max seq length	128 (src/tgt)	Source dataset is sentence-level; 99th-pctile fits
Task prefix	"paraphrase: "	Standard T5 convention

Experiment 3.2: Transformers 5.x API break

Symptom: First training run crashed at trainer construction:

```
TypeError: Seq2SeqTrainer.__init__() got an unexpected keyword argument 'tokenizer'
```

Cause: Transformers 5.0 renamed `tokenizer=` to `processing_class=` in `Trainer.__init__`. The fix is a one-character change but the error message doesn't suggest the new name.

Fix: `tokenizer=` $\rightarrow$ `processing_class=` in `src/train_t5.py:103` and `src/train_pegasus.py:113`.

Experiment 3.3: Full training run

Command: `python src/train_t5.py --data data/paraphrases --max-train-samples 100000 --epochs 3`

Duration: ~ 3 h wall-clock on M5.

Eval loss trajectory:

Step	Epoch	eval_loss
3125	1.0	1.174
6250	2.0	1.139 (best)
9375	3.0	(slight regression, not best)

The training-time loss displayed by the trainer (~5.0) was much higher than eval_loss (~1.14). This is expected — T5 has dropout enabled during training and disabled at eval. The eval signal is the reliable one.

Output artifact: checkpoints/t5-small-paraphrase/model.safetensors (231 MB).

load_best_model_at_end=True + save_total_limit=2 left only the relevant checkpoints; intermediate ones were pruned.

4. Pegasus fine-tuning attempts (failed)

Experiment 4.1: Tokenizer load failure

Goal: Fine-tune tuner007/pegasus_paraphrase for comparison.

Symptom: AutoTokenizer.from_pretrained() failed with SentencePieceExtractor requires the protobuf library, then fell back to tiktoken which also wasn't installed.

Fix: pip install protobuf. Pegasus tokenizers require it for legacy SentencePiece model conversion in transformers 5.x.

Experiment 4.2: MPS OOM on full fine-tune

Config: batch 2, grad-accum 8, max-source-len 128, no gradient checkpointing.

Symptom: Crashed at step 13/2500:

```
RuntimeError: MPS backend out of memory (MPS allocated: 11.97 GiB,
other allocations: 18.01 GiB, max allowed: 30.19 GiB).
```

Per-step latency observed: ~10.85 s/iter → would project to ~7.5 h per epoch even if memory were sufficient.

Experiment 4.3: Aggressively reduced config

Changes applied:

Knob	Before	After
max-source-len	128	96 (attention is $O(\text{seq}^2)$)
batch-size	2	1
grad-accum	8	16 (effective batch unchanged)

epochs	2	1
max-train-samples	20,000	8,000
gradient checkpointing	off	on

Symptom: Still OOM, this time at step 24/500. MPS allocated 10.21 GB, other allocations 19.84 GB.

Diagnosis: The "other allocations" 20 GB is mostly the model itself — 570M params × 4 bytes (fp32) = 2.27 GB for weights, AdamW optimizer state ~4.5 GB, gradients 2.27 GB, plus framework overhead. With 24 GB total unified memory and macOS taking some, there's no headroom for activations.

Decision: abandon fine-tuning. Used off-the-shelf `tuner007/pegasus_paraphrase` in the comparison notebook as a quality baseline instead.

Takeaway: For seq2seq fine-tuning on a 24 GB Mac, **~300M parameters is roughly the upper bound** for fp32 + AdamW. Bigger models need either:

- A CUDA box, or
- 8-bit optimizer (not supported on MPS), or
- LoRA/QLoRA (different problem class)

5. Inference pipeline development

Experiment 5.1: Sentence-level inference (v0)

File: `src/infer.py`. Single-sentence input, beam search, `num_return_sequences=N`.

Validation: Worked correctly on simple inputs. Hit the assessment's 800ms/CPU target trivially at sentence scale.

Experiment 5.2: Naive paragraph pipeline (v1, per-sentence loop)

Architecture:

```
text → split paragraphs → split sentences → for each sentence: model.generate() → rejoin
```

Implementation: `src/paragraph_pipeline.py` v1.

Result on cover letter (258 words, MPS):

- 15 sentences
- Length ratio: **1.03** (passes)
- Latency: **11.73 s** (fails, way over 800 ms target)
- Throughput: 1.3 sentences/sec

Per-call overhead (model.generate startup + token decode) dominated.

Experiment 5.3: Batched paragraph pipeline (v2)

Change: Flatten all sentences across all paragraphs → one `model.generate()` call per batch of 16 (with padding). Length-ratio selection still per-sentence after the batch returns.

Length min/max handling: Dropped `min_length` in batched mode (it applies uniformly to all items in the batch — would penalize naturally-short sentences). Length enforcement moved entirely into `_pick_best()` which selects the longest candidate satisfying $\geq 0.8 \times$ source word count.

Result on cover letter (258 words, MPS):

- Length ratio: **1.01** (unchanged, passes)
- Latency: **3.84 s** (3.1× faster than v1)
- Throughput: 3.9 sentences/sec

Takeaway: Batched generation was the single biggest latency optimization in the project. Per-sentence beam search shares no work; batching does.

6. Quantization investigation

Experiment 6.1: PyTorch dynamic INT8 quantization

File: `src/quantize.py`. Wraps `torch.quantization.quantize_dynamic(model, {nn.Linear}, dtype=torch.qint8)`.

First-run symptom on arm64 Mac:

```
RuntimeError: Didn't find engine for operation quantized::linear_prepack
NoQEngine
```

Cause: Default quantization backend on arm64 macOS isn't set. FBGEMM is x86-only; QNNPACK is the right pick for ARM.

Fix: `torch.backends.quantized.engine = "qnnpack"` before `quantize_dynamic`. Applied in `src/paragraph_pipeline.py:151`.

Experiment 6.2: Did quantization actually help?

Hypothesis going in: INT8 should be ~2× faster on CPU, with minor quality hit. This is the conventional wisdom for deploying small models.

Tested in §7 below. Spoiler: **the conventional wisdom is wrong for this hardware + model combination.**

7. CPU latency benchmark

File: `src/bench_cpu.py`. 6 configs × 3 runs each + 1 warmup, median reported.

Input: Cover letter (258 words).

Target: <800 ms on CPU.

Config	Median latency	Peak RSS	Throughput	Length ratio	Pass?
FP32 beam=5	765 ms	919 MB	338 wps	1.01	PASS
FP32 beam=2	383 ms	939 MB	674 wps	0.97	PASS
FP32 greedy	275 ms	961 MB	938 wps	0.95	PASS
INT8 beam=5	1210 ms	1289 MB	213 wps	1.03	FAIL
INT8 beam=2	613 ms	1179 MB	421 wps	1.01	PASS
INT8 greedy	297 ms	1121 MB	870 wps	0.97	PASS

Headline finding: INT8 dynamic quantization is slower than FP32 on Apple Silicon for T5-small.

Why:

- T5-small's weights (~240 MB in fp32) fit comfortably in M5 L2/L3 cache. INT8 unlocks no bandwidth savings.
- QNNPACK INT8 dynamic quantization has per-call overhead: the activations get quantized at runtime, matmul executed, then dequantized. On a small model with relatively short sequences, this overhead exceeds the matmul savings.
- Native FP32 matmul on ARM uses NEON SIMD which is already highly optimized.

Production-config decision: FP32 + beam=5 (765 ms median, 5% under target, length ratio 1.01 — highest quality of the passing configs).

Trade-off configs documented for the report:

- Speed-max: FP32 greedy (275 ms) — would handle 400+-word inputs with margin.
- Balanced: FP32 beam=2 (383 ms) — 2× headroom over target.

8. Paragraph-level metrics

Experiment 8.1: Test passage set

Cover letter (assessment-provided, 258 words) + two synthetic complex-domain passages I authored at 200–400 word scale to stress domain vocabulary:

Passage	Domain	Words
cover_letter	career	258
legal_contract_breach	legal	267
medical_diabetes	medical	243

Stored in `src/test_passages.py` with a `dump_to_files()` CLI for the inference CLI to consume.

Experiment 8.2: Metric selection

Implemented in `src/metrics_paragraph.py`:

Metric	What it measures	Direction
Length ratio	<code>paraphrase_words / source_words</code>	Want ≥ 0.8
BLEU vs source	n-gram overlap with input	Lower = more rewording
ROUGE-L vs source	longest common subsequence	Lower = less structural copying
BERTScore F1	semantic similarity via DistilBERT embeddings	Higher = better meaning preservation
Latency (paraphrase)	end-to-end seconds	Lower = better

Why not BLEU against a reference? No human-written reference paraphrases exist for these passages. BLEU-vs-source is the standard proxy for "how much rewording happened" in single-reference paraphrase eval.

Experiment 8.3: Results

Single run on CPU FP32 (beam=5):

Passage	Length ratio	BLEU vs src	ROUGE-L	BERTScore F1	Latency
cover_letter	1.01	66.4	0.844	0.954	1.44 s
legal_contract_breach	1.04	66.3	0.821	0.967	1.12 s
medical_diabetes	1.00	65.7	0.765	0.965	2.33 s
AVG	1.02	66.1	0.810	0.962	1.63 s

Reads:

- Length: PASS. All three clear the 0.80 threshold with margin.
- Semantic fidelity: PASS. BERTScore ~ 0.96 — paraphrases mean the same thing as the source.
- Diversity: WEAKNESS. BLEU 66 is **high** — the model rewrites but stays close to the source surface. This is a model-capacity issue, confirmed in §9.

Latency footnote: The 1.6 s avg here is higher than the 765 ms reported in §7 because this run had BERTScore (DistilBERT) loaded in the same process, contending for CPU threads and cache. The `bench_cpu.py` numbers in §7 are the authoritative latency figures.

9. LLM API comparison (Gemini 2.5 Flash)

Experiment 9.1: Setup

Provider: Google Gemini 2.5 Flash via the new google-genai SDK.
Configured via: .env with GEMINI_API_KEY (loaded automatically by metrics_paragraph.py).

Prompt template:

```
Paraphrase the following passage. Preserve the original meaning exactly but
rewrite using different wording and sentence structures. Keep the output
length within 10% of the original. Return only the paraphrase, no
commentary, no markdown.

Passage:
{text}
```

Experiment 9.2: Side-by-side results

System	Length ratio	BLEU vs src ↓	ROUGE-L ↓	BERTScore F1 ↑	Latency
T5-small (fine-tuned, CPU)	1.02	66.1	0.810	0.962	1.63 s
Gemini 2.5 Flash	0.99	18.8	0.469	0.888	12.62 s

Per-passage:

Passage	T5 BLEU	Gemini BLEU	T5 lat	Gemini lat	Gemini ratio
cover_letter	66.4	23.5	1.44 s	13.41 s	1.08
legal	66.3	24.1	1.12 s	11.09 s	0.82 (near threshold)
medical	65.7	8.8	2.33 s	13.35 s	1.07

Cost: 1,084 input + 938 output tokens across 3 passages at \$0.10/\$0.40 per M-token → **\$0.00016/paragraph, \$0.16/1k paragraphs.**

Experiment 9.3: Reads

- **Gemini does ~3.5× more rewording** (BLEU 19 vs 66) — what most users would call "real paraphrasing." Medical passage at BLEU 8.8 is essentially fully rewritten.
- **T5 preserves meaning slightly better** (BERTScore 0.96 vs 0.89). The gap is real but small.
- **T5 is 7.7× faster on CPU** including network round-trip overhead for Gemini.
- **Gemini compressed the legal passage** to ratio 0.82 — just barely clearing the 0.80 length threshold. T5 stays comfortably above 1.0 on all three.
- **Cost is negligible at any reasonable volume** — even 1M paragraphs through Gemini is ~\$160.

Pareto framing (no single winner):

- T5 wins: latency, length safety margin, fidelity, offline operation.

- Gemini wins: surface diversity (the actual point of paraphrasing).

10. API deployment

File: `src/server.py` — FastAPI with three endpoints:

Endpoint	Purpose
GET <code>/health</code>	Model name, device, quantized flag, default beams
GET <code>/passages, /passages/{id}</code>	Access the bundled test fixtures
POST <code>/paraphrase</code>	Body: <code>{"text": "...", "num_beams": 5, ...}</code> → paraphrase + metrics

Smoke test: Server boots in ~5 s. Cover-letter prefix sent via curl returned a paraphrase in **791 ms** end-to-end (loaded T5 on CPU, FP32, beam=5). `target_800ms_met=true` in the response payload.

Configuration via `.env`: `CPG_MODEL`, `CPG_DEVICE`, `CPG_QUANTIZED`, `CPG_NUM_BEAMS`.

11. Cumulative experiment table

#	Experiment	Outcome	Key number
1.1	Native arm64 Python	PASS - Fixed import errors	—
1.2	Venv outside iCloud	PASS - Stable installs	—
2.1	Dataset flatten	PASS - 419k → 2.1M pairs	—
3.3	T5-small full fine-tune	PASS - Production model	eval_loss 1.139
4.2	Pegasus fine-tune (default config)	FAIL - MPS OOM	crashed step 13/2500
4.3	Pegasus fine-tune (minimal config)	FAIL - MPS OOM	crashed step 24/500
5.2	Per-sentence pipeline v1	PARTIAL - Correct but slow	11.73 s, MPS
5.3	Batched pipeline v2	PASS - 3.1x speedup	3.84 s, MPS
6.1	INT8 quantization w/ QNNPACK	PASS - Runs	—
7	CPU benchmark (6 configs)	PASS - FP32 wins	765 ms
7	INT8 hypothesis	FAIL - INT8 slower than FP32	1210 ms vs 765 ms
8.3	Paragraph metrics on 3 passages	PASS - All clear 0.8 ratio	BERTScore 0.962

9.2	T5 vs Gemini comparison	PASS - Pareto trade-off	BLEU 66 vs 19
10	FastAPI deployment	PASS - End-to-end via curl	791 ms

12. Lessons learned

Verify the platform before installing anything. `platform.machine()` inside Python is the source of truth on macOS. An `x86_64` Python under Rosetta will silently get old torch wheels.

iCloud Drive and venvs are incompatible. Conflict-rename ghost files silently break installs in ways that look like pip bugs. Move venvs to `~/ .venvs/` and symlink.

Quantization conventional wisdom doesn't transfer to Apple Silicon for small models. FP32 + NEON SIMD beats torch dynamic INT8 + QNNPACK for T5-small. Always benchmark; don't assume.

Batched generation > per-call generation by a large factor. 3.1× speedup with zero quality loss. Should be the default for any seq2seq pipeline handling multiple inputs.

MPS has hard memory limits on Apple Silicon. 24 GB unified memory → ~30 GB MPS allocation ceiling (with macOS overhead) → ~300M-param fp32 models with AdamW are the practical fine-tuning ceiling. Above that, use CUDA or LoRA.

The right metric depends on whether you want fidelity or diversity. BLEU/ROUGE-L vs source measure diversity (lower = better). BERTScore measures fidelity (higher = better). They can both be high simultaneously — that's the "conservative rewording" failure mode we observed on T5-small.

Off-the-shelf can be a legitimate baseline. When fine-tuning Pegasus failed, using `tuner007/pegasus_paraphrase` as-is for comparison was a defensible and informative choice; it would have been worse to spend more time forcing fine-tuning to work.

Single-reference paraphrase eval is hard. Without human-written references, BLEU-vs-source is a proxy for "how much rewording happened" — a useful signal but not "is this paraphrase good." Multi-reference eval or human ratings would be more rigorous.

Smoke-test data preparation before training. A 30-second smoke run with `--max-rows 100` saved hours when it caught the "stringified list" issue in the `chatgpt-paraphrases` column.

Pre-installing all anticipated dependencies in one shot prevents pip churn. Each `force-reinstall` is a chance for files to get corrupted (especially under iCloud). One large `pip install` of everything is safer than incremental adds.